制御教育教材 StampFly Ecosystem の紹介

コーディングから飛行試験、データ取得まで（要点版）

伊藤 恒平（金沢工業大学）

2026 年 9 月 10 日（木）大阪大学中之島センター + Zoom
システム制御情報学会・計測自動制御学会 チュートリアル講座 2026

この資料は CC BY 4.0（クリエイティブ・コモンズ 表示 4.0 国際）で公開

https://creativecommons.org/licenses/by/4.0/deed.ja

StampFly Ecosystem とは

制御なしでは飛べない機体で、一人 1 機、「設計 → 実装 → 実験 → 解析」を一周する

マルチコプタは制御系が止まれば即座に墜落する。この性質を教材にして、フィードバック制御の役割を自分の手で確かめる

今日使うもの	役割
機体とコントローラ	手のひらサイズの 4 モータ機（ESP32 搭載）と、ESP-NOW（Espressif 社の無線直接通信方式）で操縦する送信機
sf CLI	ビルド・書き込み・ログ取得・解析をまとめた `sf` コマンド（CLI: 文字でコマンドを打って操作する方式）
SILS	Software In the Loop Simulation。実機に書き込むのと同じファームウェアのソースを PC で動かす、物理シミュレータとの閉ループ試験（Step 5 で使う）

上の道具で、センサを読むところから姿勢制御の設計まで、6 ステップで一周する

今日の流れ: 6 ステップ



準備（実習 1: 環境導入・機体の初期設定・ペアリング）のあと、番号順に進む。
Step ごとに「やること」と「知っておくこと」だけを示す

実習ごとの役割: 誰が何をするか

実習	内容	役割
1	初期設定・ペアリング	各自が自分の機体に書き込み、設定する
2, 3, 4	IMU、モータ、ミキサ	各自がコードを書き、自分の機体に書き込む（3, 4 は机上でモータを回す）
5, 6	P 制御、ゲイン設計	各自がコードを書いてビルドまで。**飛行は講師**
7	システム同定	**講師が飛ばして取ったログ**を配布し、各自が解析する
8	PID	各自がコードを書いてビルドし、**自分の PC で SILS 確認**。実機の飛行は講師
9	姿勢推定	各自がコードを書き、自分の機体に書き込む（飛ばさない）

飛行は安全のため講師だけが行う。自分のコードの飛行は、帰宅後に自分の機体で試す

機材と安全

持ち物

- ノート PC
- StampFly 機体
- コントローラ
- USB ケーブル

安全ルール

- 飛行は**講師だけ**が指定エリアで行う。参加者は距離を保つ
- モータを回す実習（実習 3, 4）は**机上**で行い、プロペラとガードに手を近づけない
- **ARM**（モータを回せる状態にすること）は機体ボタンの単クリックかコントローラで行う。**DISARM**（停止）は機体ボタンの再クリック。異常な音・振動・回転があれば即 DISARM
- ARM は USB 給電中とペアリング中は拒否される。書き込み後は USB を外し、水平な机に置いて電池で動かす
- 実習 3, 4 では、コントローラの左ボタンの高度モードを OFF（ALT/POS にしない）、右ボタンの制御モードを STABILIZE か ACRO にしておく（ACRO: 角速度だけを制御するモード、STABILIZE: 姿勢角を制御するモード）。ALT/POS では ARM 直後に自動離陸に入り、ボタンで DISARM できない
- 機体の真上に顔や手を出さない

準備 ①: 前提ツールの導入（OS ごとに違う）

OS	コマンドを打つ場所	① 前提ツールの導入
Windows	コマンドプロンプト （WSL は不要）	`winget install Git.Git`
macOS	ターミナル	Homebrew を入れ、`brew install cmake ninja dfu-util ccache`
Linux	ターミナル	`sudo apt install -y git curl tar cmake ninja-build wget flex bison gperf ccache libffi-dev libssl-dev dfu-util libusb-1.0-0`

- エディタ VSCode と、その拡張 Teleplot（`alexnesnes.teleplot`。実習 2 のグラフ表示に使う）も入れておく
- OS による違いは、導入・有効化コマンドの名前とシリアルポート名（`COM3`、`/dev/cu.usbmodem*` など）だけ。`sf` コマンド以降は共通
- 詳しくは README「インストール」節。ダウンロードが大きいので前日までに済ませる

準備 ②③: 取得・導入と、有効化・診断

② 取得と導入

```
git clone https://github.com/M5Fly-kanazawa/
    stampfly_ecosystem.git
cd stampfly_ecosystem
./install.sh
```

③ 有効化と診断

```
source setup_env.sh
sf doctor
```

Windows は `install.bat`、`setup_env.bat`（`./` も `source` も付けない）。`install` は専用の Python と ESP-IDF（Espressif 社の ESP32 向け開発環境）を自動で入れる

`sf doctor` の出力（一部）

```
[OK]   Python 3.12.6
[OK]   Dedicated environment: ~/.stampfly
[OK]   Vehicle firmware: .../firmware/vehicle
[OK]   ESP-IDF: v5.5.2
[OK]   numpy
...
[INFO] Checking serial ports...
  /dev/cu.usbmodem14101
```

[OK] が並び、最後の Summary に `[WARN]` が出なければ準備完了（機体をつなぐとシリアルポートが表示される）

実習 1 (1/2): 書き込みと機体の初期設定

目的 実習用ファームを機体に書き込み、機体ごとの初期設定を 1 回済ませる

手順（機体を USB でつなぐ）

1. `sf lesson switch sci2026:1`
2. `sf lesson build`（初回は数分）
3. `sf lesson flash`。書き込み後に**自動でモニタに入る**（機体のログの画面。`Ctrl+]` で抜ける）
4. モニタで右のコマンドを入力（機体への命令 = 機体の CLI）

モニタで機体に入力

```
param reset
param set wifi.mode 1
param set wifi.channel N
param save
reboot
```

`reset`: 初期値に戻す。`wifi.mode 1`: 機体が WiFi を出す（Step 4 で使う）。`N`: 1 / 6 / 11（講師が指定）。`save`: 保存。`reboot`: 再起動

期待結果 起動音が鳴り、モニタに `Hello StampFly!` が出る。LED は未ペアのため青の速い点滅（ペアリングすると緑）

実習 1 (2/2): コントローラと機体をペアリングする

目的 自分のコントローラと機体を 1 対 1 で結ぶ（初回のみ。以降は電源投入で自動再接続）

手順

1. **コントローラ**: M5 ボタン（画面の押し込みボタン）を押しながら電源を入れる。`=== PAIRING ===` が出る
2. **機体**: 青の速い点滅（ペアリング待ち）になっていることを確認する。未ペアの機体は自動でこの状態になる。ペア済みの機体は、本体ボタンを **3 秒以上**押し続けてペアリング待ちにする
3. **コントローラ**: 候補一覧（例 `A1B2 CH06`）から、MAC 下 4 桁（機体の固有番号の末尾 4 桁）が機体のラベルと同じ行を右スティックで選び、M5 ボタンで確定する

取り違えに注意

複数組が同時に行うため、候補が 1 件でも確定操作は省けない。ラベルが無ければ、機体を USB でつなぎ、`sf monitor` に `mac` と入力すると下 4 桁が分かる

期待結果 機体の LED が**緑**に変わればペアリング成立（画面はフライト画面になる）

書くのは 2 関数だけ。作業は同じコマンドの繰り返し

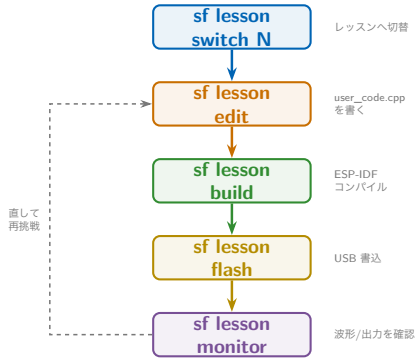
書く関数（実習コード = user_code.cpp）

- `setup()` — 起動時に 1 回
- `loop_400Hz(dt)` — 400 Hz（2.5 ms）ごと。dt は周期 [s]

センサやモータの詳細は `ws::` で始まる実習用の関数（ws は workshop 用 API の名前空間）が隠す

作業サイクル（実習番号 N）

```
sf lesson switch sci2026:N
sf lesson edit / build / flash / monitor
```



`sf lesson` は実習用ファーム（`firmware/workshop`）を扱い、実習 1〜9 はこれ 1 種類で通す。`switch` は雛形を `user_code.cpp` に写す（編集済みの内容は `my_code/` に退避）。解答は `--solution`

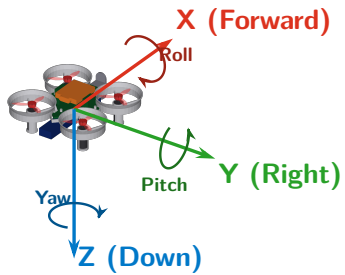
Step 1

センサ — 機体の動きを数値で見る



SCI/SICE チュートリアル講座 2026（要点版）

機体の軸と IMU の API



IMU（慣性計測装置）は BMI270（3 軸ジャイロ + 3 軸加速度計）で、400 Hz で値が更新される。軸は機体座標系 FRD（前方 - 右 - 下）に合わせてある

関数	意味	単位
`ws::gyro_x/y/z()`	角速度	rad/s
`ws::accel_x/y/z()`	加速度	$\mathrm{m/s^2}$

静止時の目安: ジャイロ ≈ 0、加速度 Z $\approx -9.81\,\mathrm{m/s^2}$（Z 軸は下向きが正なので、重力に対する反力は負になる）

実習 2: IMU の値を見る

目的 ジャイロと加速度の値を画面にグラフで出し、軸と符号を確かめる

手順

1. `sf lesson switch sci2026:2`
2. `sf lesson edit` で右のコードにして保存
3. `sf lesson build`、`sf lesson flash`
4. VSCode の Teleplot で、>名前: 値 の行をグラフにする

user_code.cpp

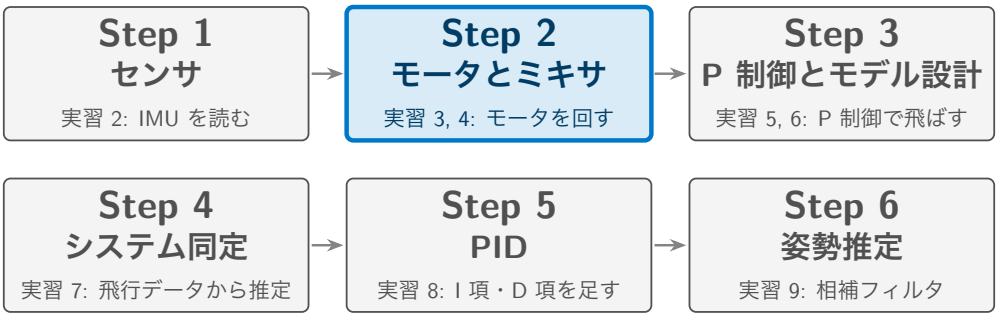
```cpp
static uint32_t tick = 0;
void loop_400Hz(float dt) {
  if (tick++ % 4 == 0) {  // 100Hz
    ws::print(">gx:%.3f",ws::gyro_x());
    ws::print(">gy:%.3f",ws::gyro_y());
    ws::print(">az:%.2f",ws::accel_z
        ());
  }
}
```

期待結果 機体を傾けると gx（ロール）、gy（ピッチ）が傾ける速さに応じて振れる。静止すると 0 付近、az は -9.81 付近

詰まったら `sf lesson switch sci2026:2 --solution`（解答に切り替える）

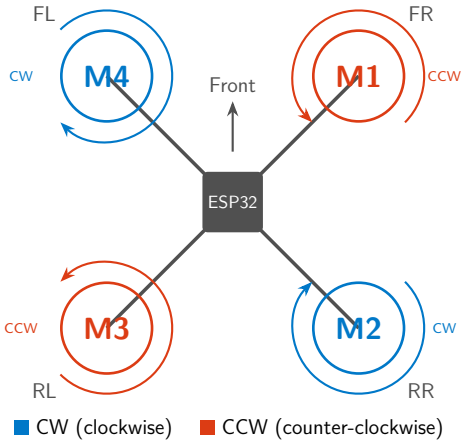
Step 2

モータとミキサ — 機体を動かす側を作る



SCI/SICE チュートリアル講座 2026（要点版）

モータの配置と Duty



- モータ ID: 1 = FR（右前）、2 = RR（右後）、3 = RL（左後）、4 = FL（左前）
- 対角のモータが同じ向きに回り、反作用トルクを打ち消す（M1・M3 = CCW、M2・M4 = CW）
- モータは PWM（パルス幅変調: 電圧を入り切りする時間の割合で平均電圧を変える方式）の Duty 比で駆動する。Duty 0 = 停止、1 = 最大
- API: `ws::motor_set_duty(id, duty)`（id = 1–4、duty = 0.0–1.0）

実習 3: Duty を決め打ちして 1 つのモータを回す

目的 Duty とモータ ID の対応を、回る場所で確かめる

手順（机上）

1. `sf lesson switch sci2026:3`
2. `sf lesson edit` で右のコードにして保存
3. `sf lesson build`、`sf lesson flash`
4. USB を外し、水平な机に置いて、機体ボタンを 1 回クリックして ARM

user_code.cpp

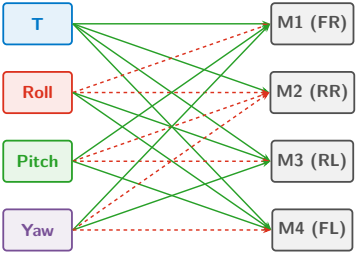
```
1 void loop_400Hz(float dt) {
2     ws::motor_set_duty(1, 0.10f);
3     ws::motor_set_duty(2, 0.00f);
4     ws::motor_set_duty(3, 0.00f);
5     ws::motor_set_duty(4, 0.00f);
6 }
```

Duty は 0.15 以下（ファームは制限しない）。`ws::arm()` は書かず、ボタンで ARM。ALT/POS は OFF（前々ページの安全ルール）

期待結果 FR（右前）のモータだけが回る。止めるときはボタンを再クリック（DISARM）

詰まったら `sf lesson switch sci2026:3 --solution`（解答に切り替える）

ミキサ: 推力とトルク指令を 4 つの Duty に配る



$$M_1 = T + \tfrac{0.25}{3.7}(-R + P + Y)$$
$$M_2 = T + \tfrac{0.25}{3.7}(-R - P - Y)$$
$$M_3 = T + \tfrac{0.25}{3.7}(+R - P + Y)$$
$$M_4 = T + \tfrac{0.25}{3.7}(+R + P - Y)$$

— + positive
- - - − negative

- 入力: 推力 $T \in [0, 1]$、ロール・ピッチ・ヨーのトルク指令 $R, P, Y \in [-1, 1]$
- 出力: 4 モータの Duty。各式の符号はモータ配置に対応する（例: ピッチ + = 機首上げのとき、前の M1・M4 を強め、後ろの M2・M3 を弱める）
- API: `ws::motor_mixer(T, R, P, Y)`

$$d_{\mathrm{FR}} = T + \frac{0.25}{3.7}(-R + P + Y)$$

他のモータは符号の組だけが違う。係数 $0.25/3.7$ は、旧世代ファームのミキサから引き継いだ調整定数（物理量ではなく経験的なスケール）

実習 4: スティックでモータを回す

目的 スティック指令がミキサを通って、モータの回転差になることを確かめる

手順（机上）

1. `sf lesson switch sci2026:4`
2. `sf lesson edit` で右のコードにして保存
3. `sf lesson build`、`sf lesson flash`
4. USB を外して机に置き、ARM。コントローラで操作する

user_code.cpp

```
1 void loop_400Hz(float dt) {
2     float t = ws::rc_throttle();
3     float r = ws::rc_roll();
4     float p = ws::rc_pitch();
5     float y = ws::rc_yaw();
6     ws::motor_mixer(t, r, p, y);
7 }
```

スロットルは Duty に直結。15% 以下に保つ。ALT/POS は OFF

期待結果 スティックを倒した向きに応じて、モータの回転に差が出る

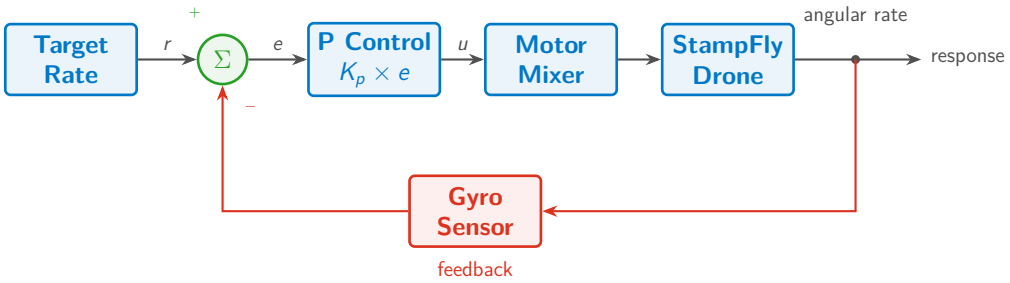
詰まったら `sf lesson switch sci2026:4 --solution`（解答に切り替える）

Step 3
P 制御とモデルに基づく設計



SCI/SICE チュートリアル講座 2026（要点版）

これだけでは飛べない: フィードバックと内側ループ



- 実習 4 の開ループ（スティック → ミキサ → モータ）では、モータの個体差、機体の非対称（推力軸・重心のずれ）、電池電圧の低下に何の補正もかからない
- そこで角速度をジャイロで測り、目標との差（誤差 $e = r - \omega$）に比例ゲイン K_p を掛けてミキサへ戻す。これを**内側ループ（レート制御）**と呼ぶ
- 姿勢角の制御はこの外側に重なる（STABILIZE 以上のモード）。今日は内側ループだけを扱う

実習 5: レート P 制御で飛ばす

目的 ジャイロでフィードバックをかけ、制御の効果を見る（K_p は勘で決めた値）

手順

1. `sf lesson switch sci2026:5`
2. `sf lesson edit` で TODO を埋めて保存
3. `sf lesson build`（ここまで各自）
4. 飛行は講師が完成コードで行う

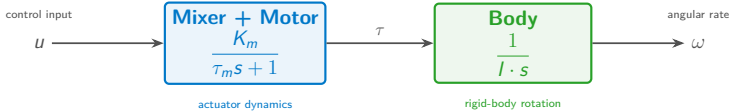
user_code.cpp（赤字を書く）

```
Kp_rp = 0.5f; Kp_yaw = 2.0f;
float rt = ws::rc_roll() * rate_max_rp;
float re = rt - ws::gyro_x();
float ro = Kp_rp * re;
// pitch, yaw: same form
ws::set_rate_target(rt, pt, yt);
ws::motor_mixer(ws::rc_throttle(),
                ro, po, yo);
```

期待結果 スティックに機体が追従し、わずかな振動が見える

詰まったら `sf lesson switch sci2026:5 --solution`（解答に切り替える）。コード欄の**赤字**が自分で書く TODO の部分

プラントの伝達関数



$$G_p(s) = \frac{K}{s(\tau_m s + 1)}, \qquad K = \frac{K_m}{I}$$

ホバー近傍で線形化すると 2 ブロックの直列になる

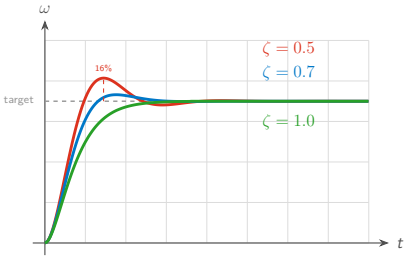
- ミキサ + モータ: Duty → トルク、1 次遅れ $K_m/(\tau_m s + 1)$
- 機体: トルク → 角速度、積分器 $1/(Is)$（I: 慣性モーメント）
- 直列にすると $G_p(s) = \dfrac{K}{s(\tau_m s + 1)}$、$K = K_m/I$

実測パラメータ

パラメータ	記号	Roll	Pitch	Yaw
慣性モーメント	I	9.16e-6	13.3e-6	20.4e-6 kg·m^2
トルクゲイン	K_m	9.3e-4	9.3e-4	1.67e-4 N·m
プラントゲイン	$K = K_m/I$	**102**	**70**	$\approx$**8** rad/s^2

- モータ時定数 τ_m は 0.02 s を使う（3 軸共通。実測諸元からの計算値は 0.0163 s）
- Yaw の K は計算で 8.2、実習コードは 8.0 を使う。Roll/Pitch より 1 桁小さい（ヨートルクを出しにくい）ので、次ページの設計式では軸ごとに必要な K_p が大きく違う

閉ループの 2 次系と、ζ から K_p を決める式



P 制御の閉ループは 2 次系になる（ω_n: 固有角周波数 [rad/s]、ζ: 減衰比）:

$$G_{cl}(s) = \frac{K_p K}{\tau_m s^2 + s + K_p K}$$

$$\omega_n = \sqrt{\frac{K_p K}{\tau_m}}, \quad \zeta = \frac{1}{2\sqrt{K_p K \tau_m}}$$

狙う ζ から K_p を逆算する設計式:

$$\boxed{K_p = \frac{1}{4\zeta^2 K \tau_m}}$$

オーバーシュート量 $\approx e^{-\pi\zeta/\sqrt{1-\zeta^2}}$（$\zeta = 0.5 \to 16\%$、$\zeta = 0.7 \to 5\%$、$\zeta \geq 1 \to 0\%$）

実習 6: 設計式で K_p を計算して飛ばす

目的 軸ごとに設計式から K_p を求め、ζ を変えたときの違いを見る

手順

1. `sf lesson switch sci2026:6`
2. `sf lesson edit` で `Kp_*` を設計式で計算して保存
3. `sf lesson build`（ここまで各自）
4. 飛行は講師が $\zeta = 0.7 \to 0.5 \to 1.0$ の順に変えて行う

user_code.cpp（赤字を書く）

```
tau_m = 0.02f;  K_roll = 102.0f;
K_pitch = 70.0f;  K_yaw = 8.0f;
float zeta = 0.7f;
float c = 1.0f / (4*zeta*zeta * tau_m);
float Kp_roll  = c / K_roll;
float Kp_pitch = c / K_pitch;
float Kp_yaw   = c / K_yaw;
```

期待結果 $\zeta = 0.5$ は行き過ぎが大きく、$\zeta = 1.0$ は反応が鈍い

詰まったら `sf lesson switch sci2026:6 --solution`（解答に切り替える）。コード欄の**赤字**が自分で書く TODO の部分

実習 6 の結果: 軸ごとに K_p は違う

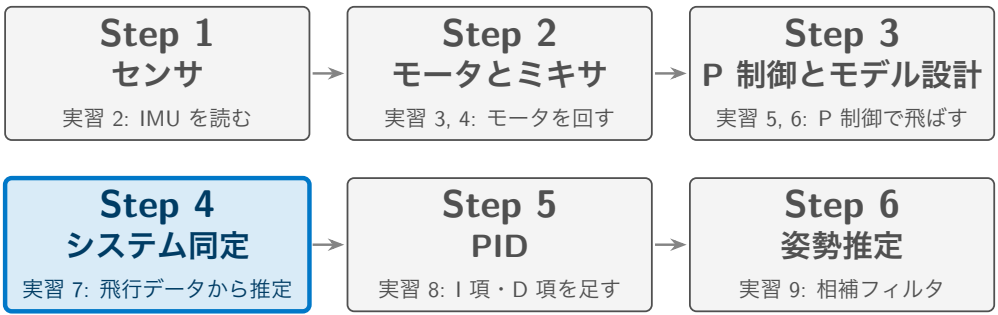
	K	実習 5 の K_p	その ζ	$\zeta = 0.5$ の K_p	$\zeta = 0.7$ の K_p	$\zeta = 1.0$ の K_p
Roll	102	0.5	0.50	0.49	0.25	0.12
Pitch	70	0.5	0.60	0.71	0.36	0.18
Yaw	8.0	2.0	0.88	6.25	3.19	1.56

読み取り

実習 5 の勘の値は、Roll で $\zeta \approx 0.5$、Pitch で 0.6（やや振動的）、Yaw は $K_p = 2.0$ で $\zeta \approx 0.88$ だった。K が軸ごとに 1 桁違うため、同じ K_p を 3 軸に使い回せない。モデルがあれば、勘の値の良し悪しを数値で言える

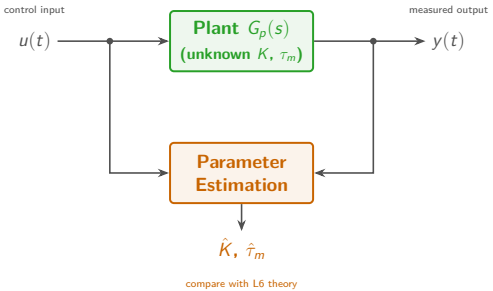
Step 4

システム同定 — 飛行データから推定する



SCI/SICE チュートリアル講座 2026（要点版）

システム同定: 飛行データから K, τ_m を推定する



図中の L6 = 実習 6 の理論値

入力と出力の記録から、プラント $G_p(s) = K/(s(\tau_m s + 1))$ の K, τ_m を求める。一般には、候補の K, τ_m でモデルの出力 $\hat{y}(t)$ を計算し、実測 $y(t)$ との二乗誤差が最小になる値を探す（最小二乗フィット）：

$$J(K, \tau_m) = \sum_t \big(\hat{y}(t) - y(t)\big)^2$$

`sf sysid fit` は既定で、閉ループ（P 制御で飛ばした記録）の「目標角速度 $\rightarrow$ ジャイロ」の応答にモデルを当てはめる。制御なしでは飛べない機体なので閉ループで同定する。K_p は記録のモータ Duty から自動で推定する

データの取り方: 機体の WiFi につなぐ

PC を機体の WiFi につなぐ

- 実習 1 の `wifi.mode 1` で、機体が自分の WiFi を出している
- SSID は `StampFly-XXYY`（XXYY = 機体の MAC アドレス末尾 4 桁。機体のラベルと同じ）、パスワードは `stampfly`
- 機体の IP は `192.168.10.1`。接続中、PC は会場のネットから切り替わる

方法	使う場面
Teleplot（USB）	その場で波形を見る（実習 2）
`sf telemetry`（WiFi）	コード変更なしで 50 Hz の状態を見る
`sf log wifi`（WiFi）	後で解析するため 400 Hz で記録する

WiFi は飛行中も使える（USB は飛行中に挿せない）。今日の実習 7 では `sf log wifi` を使う

実習 7: ログを解析し、K, τ_m を求める

目的 実習 6 の理論値 K, τ_m が、実機の飛行データと一致するか確かめる

手順

1. `sf lesson switch sci2026:7`、`sf lesson build`（TODO は `ws::set_rate_target` の呼び出し。目標角速度をログに記録する。実習 5 の雛形にも同じ TODO がある）
2. **講師**が飛ばし、`sf log wifi -d 30` で記録する。ログは配布される
3. **各自**: 配布されたログ（自分で取ったログでもよい）に `sf sysid fit`

コマンド

```
sf log list
sf sysid fit <ログ名> --plot
```

`.sflog.zip` は信号ごとの CSV をまとめた zip（フライトログ一式）。`sf log wifi` は `logs/flight_<日時>.sflog.zip` に保存し、`sf log list` で名前を確認できる

期待結果 軸ごとの K, τ_m と当てはまりの良さ R^2 が表示され、実習 6 の理論値と比べられる

詰まったら `sf lesson switch sci2026:7 --solution`（解答に切り替える）

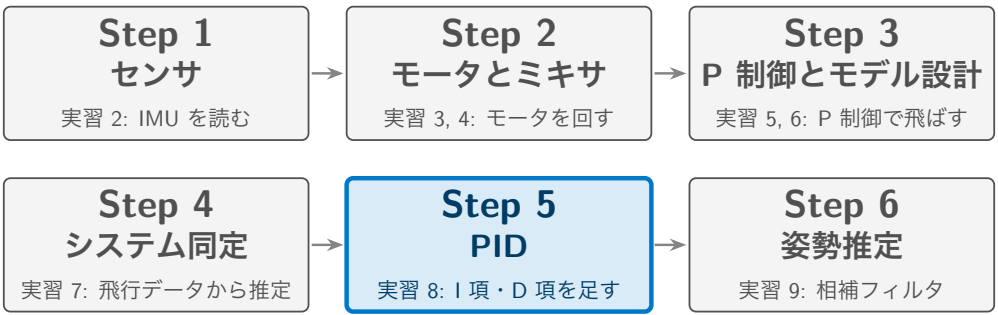
同定結果と理論値の比較（例）

軸	K（同定）	K（理論）	τ_m（同定）	τ_m（理論）	R^2
Roll	98.5	102.0	0.019 s	0.020 s	0.94
Pitch	72.1	70.0	0.021 s	0.020 s	0.91

上の数値は読み方を示すための例（説明用の数値）で、実測ではない。実際の値は飛行ごとに変わる

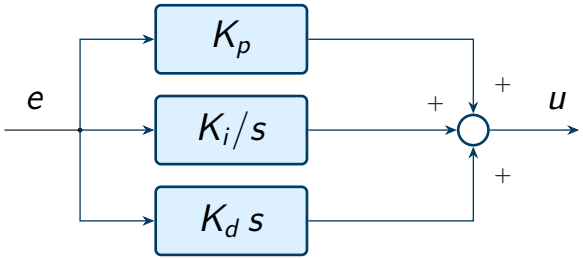
- K が理論値の数 % 以内で一致すれば、実習 6 のモデルは実機を説明できている
- R^2 は 1 に近いほど、モデルがデータをよく説明している
- 人がスティックで操縦した記録は励振（入力で系を十分に動かすこと）が小さいので、当てはまりが悪いときは別のログで試す

Step 5
PID — I 項・D 項を足す



SCI/SICE チュートリアル講座 2026（要点版）

PID 制御器の構成（並列形）



$$C(s) = K_p + \frac{K_i}{s} + K_d\,s$$

e: 誤差（目標角速度 − ジャイロ）、
u: ミキサへの指令

実習 8 の解答のゲイン

	K_p	K_i	K_d
Roll	0.25	0.3	0.005
Pitch	0.36	0.3	0.005
Yaw	2.0	0.5	0.01

Roll/Pitch の K_p は $\zeta = 0.7$ の設計値、Yaw は $\zeta \approx 0.88$ 相当

- **P**: 今の誤差に比例して押し戻す。一定の外乱トルク（重心のずれ・推力差）に対しては偏差が残る
- **I**: 誤差の積み重ねで押す。一定の外乱に対する偏差を消す。積分値を ± 0.5 で打ち切る（アンチワインドアップ: 飽和中に積分が溜まり続けるのを防ぐ）
- **D**: 誤差の変化の速さに比例して押す。行き過ぎ（オーバーシュート）を抑える

実習 8: PID を実装する

目的 実習 5 の P 制御に I 項・D 項を足し、改善を確かめる

手順

1. `sf lesson switch sci2026:8`
2. `sf lesson edit` で 3 軸に I・D 項と打ち切りを足して保存
3. `sf lesson build`
4. 実機の飛行は講師が行う

user_code.cpp（Roll 軸）

```
static float integral = 0, prev_err = 0;
if disarmed: integral = prev_err = 0;
float err = target - ws::gyro_x();
integral += err * dt;
integral = clamp(integral, 0.5f);
float u = Kp*err + Ki*integral
  + Kd*(err - prev_err)/dt;
prev_err = err;
roll_out = clamp(u, 1.0f);
```

期待結果 実習 5（P のみ）より行き過ぎが抑えられる（SILS で確認）

詰まったら `sf lesson switch sci2026:8 --solution`（解答に切り替える）。コード欄の**赤字**が自分で書く TODO の部分

実習 8 を SILS で確認する

目的 書いたコードを PC 上で飛ばし、ソフトの誤り（状態遷移の抜け、発散）がないか確かめる

SILS（Software In the Loop Simulation）：実機と同じファームのソースを PC で動かし、物理シミュレータと閉ループでつなぐ。墜落の心配なく試せる

手順（全員、実機は不要）

1. 自分のコードのまま `sf lesson sils`
2. 書き切れなかった人は先に `sf lesson switch sci2026:8 --solution`

コマンド

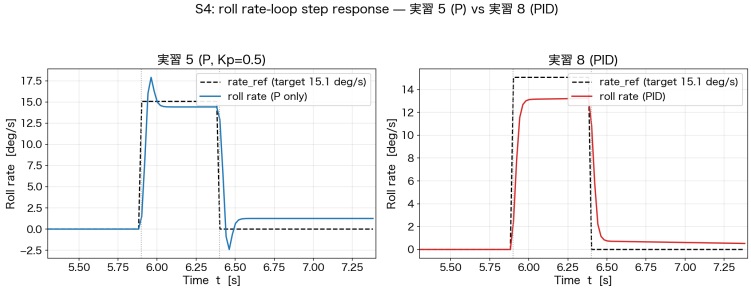
```
sf lesson sils
```

期待結果: PASS

`workshop_acro.expect` で判定: ログに ARM accepted、Takeoff detected、Takeoff complete、DISARM accepted がこの順に出る、高度 > 0.1 m、傾き $15°$ 未満

`--scenario step`: ステップ入力のシナリオ（`.expect` の合否判定はなく、応答を見る用）。`sf sils gui` は機体本体のファーム向けで、実習コードでは未確認

期待結果: P と PID のロールのステップ応答

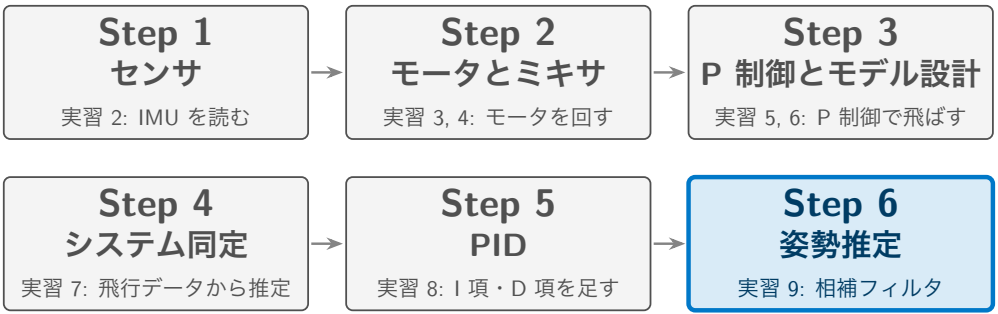


- 実習 5（P）: 目標 15.1 deg/s に対しピーク 17.9 deg/s の行き過ぎ。指令を戻すと -2.4 deg/s まで振れ、$+1.3$ deg/s が残る
- 実習 8（PID）: 行き過ぎや振動がなく、立ち上がりはゆるやか

SILS（MuJoCo）、ロール指令 $+0.3$ を 0.5 s。再現: `sf lesson switch sci2026:8 --solution`、`sf lesson sils --scenario step`

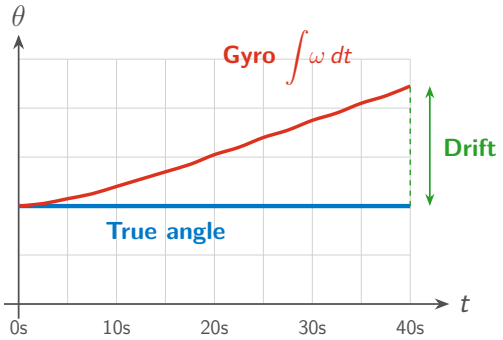
Step 6

姿勢推定 — ジャイロと加速度を混ぜる



SCI/SICE チュートリアル講座 2026（要点版）

ジャイロだけでは姿勢角が求まらない

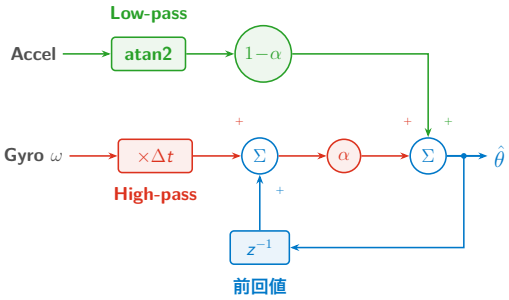


- ジャイロは角速度 ω を測る。角度は積分 $\theta = \int \omega\, dt$ で求める
- 測定のバイアス（偏り）も積分されるので、時間とともに誤差が増える（ドリフト）
- 姿勢角そのものを目標にする制御（STABILIZE 以上）には、角度の推定が要る

そこで

重力の向きを測る加速度計で角度を補う。加速度計は長期で安定だが、短期は振動や加減速でノイズが乗る

相補フィルタ: ジャイロと加速度計を混ぜる



静止時の加速度計は重力の反力を測るので、傾きが求まる（ϕ: ロール、θ: ピッチ）：

$$\phi_a = \operatorname{atan2}(-a_y, -a_z)$$

$$\theta_a = \operatorname{atan2}\left(a_x, \sqrt{a_y^2 + a_z^2}\right)$$

相補フィルタ（$\alpha = 0.98$）：

$$\hat{\theta}_k = \alpha(\hat{\theta}_{k-1} + \omega\,\Delta t) + (1-\alpha)\theta_a$$

短期はジャイロ、長期は加速度計を使う（互いの弱点を補う）。ヨー角は重力から求まらない

実習 9: 相補フィルタを実装し、機体の推定と比較べる

目的 自作の相補フィルタの角度を、機体既定の推定器 ESKF（誤差状態カルマンフィルタ）と比べる

手順（飛行しない）

1. `sf lesson switch sci2026:9`
2. `sf lesson edit` で右のコードにして保存
3. `sf lesson build`、`sf lesson flash`
4. `sf lesson monitor`、Teleplot で `cf_roll` と `eskf_roll` を重ねる

user_code.cpp（赤字を書く）

```
float a_roll = atan2f(-ay, -az);
float a_pitch = atan2f(ax,
    sqrtf(ay*ay + az*az));
cf_roll = alpha*(cf_roll + gx*dt)
        + (1-alpha)*a_roll;
// pitch: same form (gy, a_pitch)
ws::print(">cf_roll:%.2f", ...);
ws::print(">eskf_roll:%.2f", ...);
```

期待結果 機体を手で傾けると 2 つの角度がほぼ一致して動く。α を変えて違いを見る

詰まったら `sf lesson switch sci2026:9 --solution`（解答に切り替える）。コード欄の**赤字**が自分で書く TODO の部分

今日やったこと

Step	実習	得たもの
1 センサ	2	軸と符号、IMU の値の見方
2 モータとミキサ	3, 4	Duty、ミキサ、開ループでは飛べない理由
3 P 制御とモデル	5, 6	$G_p = K/(s(\tau_m s + 1))$、$K_p = 1/(4\zeta^2 K\tau_m)$
4 システム同定	7	飛行データから K, τ_m を推定し、理論値と照合
5 PID	8	I 項・D 項の役割、SILS での確認
6 姿勢推定	9	相補フィルタ（ジャイロ + 加速度計）

設計 → 実装 → 実験 → 解析を、1 機で一周した

この先へ

したいこと	入口
自分の制御器・推定器を載せる	`sf app new <名前>`。入門は `docs/guides/custom_program.md`
シミュレータをブラウザで触る	`sf sils build`、`sf sils gui`
位相余裕・むだ時間、不完全微分、周波数同定、自動チューニングなど	現行版スライド `slides/sci_tutorial.pdf` の第 4・5 セッション
復習の手順・コマンド・API の早見表	`handson_guide.md`、`cheatsheet.md`（同じフォルダ）

リポジトリ: `https://github.com/M5Fly-kanazawa/stampfly_ecosystem` ／ 資料: `https://www.docswell.com/s/Kouhei_Ito/5L387D-2026-09-10-064635`

コマンド早見表

`sf doctor`	環境診断（困ったらまずこれ）
`sf lesson switch sci2026:N [--solution]`	実習 N の雛形（解答）に切り替え
`sf lesson edit / build / flash / monitor`	編集・ビルド・書き込み・モニタ
`sf lesson sils [--scenario step]`	実習コードを SILS で確認
`sf telemetry`	50 Hz の状態をライブ表示（WiFi）
`sf log wifi -d 30`	400 Hz で記録（WiFi）
`sf log list` / `sf log viz`	ログ一覧 / 最新ログの可視化
`sf sysid fit <ログ> --plot`	プラント同定

困ったとき

症状	対処
ビルドエラー	エラー 1 行目のファイル名・行番号を見て、コードの打ち間違い（カンマ、セミコロン）を直す
`sf lesson flash` が失敗	別のケーブル・ポートを試す。だめなら機体の BOOT ボタン（基板上の、書き込み専用モードに入るボタン）を押したまま USB を挿し直す
モータが回らない	ARM されていない。USB 給電中・ペアリング中は ARM が拒否されるので、USB を外し、水平な机に置く。ALT/POS モードでは自動離陸に入るので OFF にする
LED が緑にならない	起動時の校正中に動かした。静かに置き直し、緑とビープ 3 回を待つ
機体の WiFi が見えない	電源と `wifi.mode 1` を確認
`sf doctor` が通らない	表示された項目を確認し、`docs/guides/troubleshooting.md` を見る